

# 关于基于Electron跨端开发框架开发景智文档的可行性分析

浏览器报告 “**Not allowed to load local resource: file:///...**” 这个错误，正是导致图片变占位符的直接原因。

要强调的是：你无法，也不应该“解决”这个浏览器报错。

这不是一个可以通过修改配置来“修复”的 bug，而是现代浏览器一项至关重要的、内置的安全特性。它存在的目的就是为了保护用户，防止任何网站随意读取你电脑上的本地文件。如果任何网页都能加载 `file:///` 资源，那么恶意网站就可以轻松地扫描并窃取你电脑里的个人文档、照片和敏感信息。

因此，正确的解决思路不是去对抗或关闭这个安全机制，而是在这个安全机制的规则内，用更智能的方式来处理粘贴操作。

## 真正的解决方案：拦截并处理原始数据

解决方案的核心在于改变处理粘贴事件的方式。我们要在浏览器尝试加载那个无效的 `file:///` 链接之前，就介入进来，从剪贴板中直接提取出真正的图片数据。

这需要对 OnlyOffice 的前端 JavaScript 代码进行二次开发。目标位置依然是 `DocumentServer/sdkjs/` 目录下处理编辑器输入的核心代码。

以下是详细的实施步骤：

### 1. 找到并接管 `paste` 事件处理器

在 `sdkjs` 的代码中，你需要找到负责处理粘贴（`paste`）事件的函数。你可以通过在代码库中搜索关键字如 `addEventListener('paste', clipboardData, handlePaste)` 来定位相关代码。

### 2. 修改事件处理逻辑

你需要修改这个函数，或者用你自己的函数覆盖它，实现以下逻辑：

- 阻止默认行为:** 在事件处理函数的一开始，立即调用 `event.preventDefault()`。这会阻止浏览器自行处理粘贴内容，从而避免它去尝试加载 `file:///` 并显示占位符。
- 优先使用 Clipboard API 读取图片:** 使用现代的 `navigator.clipboard.read()` API。这个 API 能够访问剪贴板中更丰富的内容，包括原始的文件或二进制数据。
- 提取图片数据:** 遍历 `clipboard.read()` 返回的项目，检查是否存在类型（`item.types`）为 `image/*`（例如 `image/png`, `image/jpeg`）的数据。
- 将数据转换为 Blob:** 如果找到了图片数据，使用 `item.getType('image/png')` 将其读取为一个 `Blob` 对象。`Blob` 是一个代表二进制文件的对象。
- 上传图片到服务器:** 这是关键的一步。OnlyOffice 已经有自己的一套上传机制。你需要找到前端代码中用于上传图片（比如通过“插入”菜单上传的图片）的函数。这个函数会接收一个文件或 `Blob` 对象，通过 `FormData` 将其发送到 `DocumentServer/server/` 的某个 API 端点。你需要复用这个上传逻辑，将上一步获取的 `Blob` 发送给服务器。

6. **获取返回的 URL:** 服务器成功保存图片后，会返回一个可在网络上访问的、指向这张新图片的 URL（例如 `/doc/ab12cd.../media/image1.png`）。
  7. **插入图片到文档:** 使用 OnlyOffice 编辑器的 API（例如 `Api.InsertImage` 或类似方法），将这个由服务器返回的有效 URL 插入到当前光标位置。
  8. **实现回退机制 (Fallback):** 如果 `navigator.clipboard.read()` 执行失败（比如在旧版浏览器或用户拒绝授权），或者剪贴板里根本没有图片数据，那么你就回退到原有的处理逻辑：通过 `event.clipboardData.getData('text/html')` 获取 HTML 内容。在处理这些 HTML 时，最好再增加一步清理操作，用正则表达式等方法移除掉所有 `src` 指向 `file:///` 的 `<img>` 标签，避免它们再次产生占位符。
- 总之，解决这个问题的正确路径是**拥抱 Web 的安全模型**，通过编写更健壮的前端代码来改善用户体验。这正是二次开发的價值所在。

## 1. OnlyOffice 有没有非浏览器版本的编辑器？

有。

这个产品叫做 **ONLYOFFICE Desktop Editors** (桌面编辑器)。

- 它是一个**免费、开源**的桌面应用程序。
- 支持 **Windows、macOS 和 Linux** 三大主流桌面操作系统。
- 它的外观和功能与你在浏览器中看到的 OnlyOffice 编辑器几乎一模一样，因为它们共享同一个核心编辑引擎。
- 你完全可以把它当作一个本地的、离线的办公套件来使用，就像使用 Microsoft Office 或 WPS 一样，直接在你的电脑上创建、打开和编辑 `.docx`, `.xlsx`, `.pptx` 等格式的文件。

## 2. 非浏览器版本能否支持多人协同编辑？

能，但这正是它设计的巧妙之处。

**ONLYOFFICE Desktop Editors** 支持两种工作模式：

1. **离线模式 (Offline Mode):** 这是默认模式。你像使用任何传统办公软件一样处理本地文件。在这种模式下，**不支持实时协同编辑**，因为文件只存在于你自己的电脑上。
2. **在线模式 (Online/Connected Mode):** 这是实现协同编辑的关键。**ONLYOFFICE Desktop Editors** 应用程序可以连接到一个云端或私有部署的 **ONLYOFFICE Docs** (`DocumentServer`) 实例——也就是你目前正在研究和二次开发的项目！

工作原理如下：

- 在桌面编辑器中，你可以添加一个账户，指向你的 OnlyOffice 服务器地址。
- 连接成功后，桌面编辑器就变成了一个功能强大的“**原生客户端**”。你不再是打开本地文件，而是直接浏览和打开存储在服务器上的文档。
- 当你编辑这个文档时，所有的操作都会像在浏览器中一样，实时发送到 `DocumentServer`。

- `DocumentServer` 负责处理和合并所有协作者（无论是使用浏览器还是也使用桌面编辑器）的修改，然后再将最终结果分发给每一个连接的客户端。

## 总结

| 特性   | 浏览器版编辑器                             | 桌面版编辑器 (离线模式)                       | 桌面版编辑器 (连接到服务器)                     |
|------|-------------------------------------|-------------------------------------|-------------------------------------|
| 运行环境 | Web 浏览器                             | 本地操作系统 (Win/Mac/Linux)              | 本地操作系统 (Win/Mac/Linux)              |
| 文件来源 | ONLYOFFICE 服务器                      | 本地硬盘                                | ONLYOFFICE 服务器                      |
| 多人协同 | 支持 (核心功能)                           | 不支持                                 | 支持 (通过服务器)                          |
| 核心引擎 | <code>DocumentServer</code> (sdkjs) | <code>DocumentServer</code> (sdkjs) | <code>DocumentServer</code> (sdkjs) |

OnlyOffice 提供了一个非常灵活的混合解决方案：

- 用户可以享受原生桌面应用的性能和稳定性。
- 同时，通过连接到你部署的 `DocumentServer`，又能获得与浏览器版本完全一致的强大实时协同编辑能力。

桌面版编辑器在连接模式下是一个非常强大的客户端，其核心编辑功能与浏览器版完全一致。但是，由于其技术架构和运行环境的不同，确实存在一些限制和差异。

这些限制可以分为几个方面：

## 1. 技术架构和性能限制

- **本质是封装的 Web 应用:** `ONLYOFFICE Desktop Editors` 并非从零开始编写的原生应用，而是使用类似 Electron 的技术，将你在浏览器中看到的 Web 编辑器（`DocumentServer` 的核心）打包成一个桌面程序。
  - **资源占用:** 这意味着它在运行时需要加载一个完整的 Web 渲染引擎（类似一个精简的 Chrome）。因此，相比于纯原生编写的软件（如微软 Office 的早期版本），它的内存和 CPU 占用可能会稍高，启动速度也可能稍慢。
  - **无法完全摆脱 Web 限制:** 虽然它是一个桌面程序，但其核心编辑区域仍然是一个 Web 环境，因此某些底层的 Web 技术限制依然可能存在，只是比在浏览器沙箱中要宽松。

## 2. 功能和集成限制

- **插件生态系统:** 这是与 Microsoft Office 相比最显著的差距之一。
  - Office 拥有数十年积累的、极其庞大和成熟的 COM/VSTO 插件生态，可以实现与操作系统、其他桌面软件的深度集成。
  - ONLYOFFICE 的插件系统是基于 JavaScript 的，虽然灵活且在不断发展，但在数量、成熟度和与本地系统深度交互的能力上，还无法与 Office 匹敌。例如，一个需要调用本地硬件或特定企业软件 API 的复杂插件，在 ONLYOFFICE 中很难实现。
- **与操作系统的深度集成:**

- **对象链接与嵌入 (OLE):** 在 Windows 上, 你可以将一个 Excel 表格“嵌入”到 Word 文档中, 双击它就能用 Excel 的界面来编辑。这种深度的原生 OLE 集成在 `ONLYOFFICE Desktop Editors` 中是有限的或不支持的。
- **系统特定功能:** 某些操作系统特有的功能 (如 macOS 的 Handoff/Continuity 或 Windows 的 Timeline) 与原生应用的集成通常比与封装的 Web 应用更好。
- **字体渲染的细微差别:** 虽然 ONLYOFFICE 尽力确保跨平台的一致性, 但由于桌面版调用的是操作系统的字体渲染引擎, 而浏览器版使用的是浏览器自己的引擎, 所以在极少数情况下, 同一文档在不同客户端上可能会有极其细微的显示差异。

### 3. 操作和环境限制

- **强制的网络依赖:** 这是“连接模式”的固有“限制”。一旦连接到服务器进行协同编辑, **网络连接就成了生命线**。如果断网, 你将无法保存更改到服务器, 也无法接收他人的实时更新。当然, 它通常有本地缓存机制, 允许你在重新连接后同步, 但这期间无法进行协同。
- **版本兼容性:** 桌面编辑器的版本需要与你的 `DocumentServer` 服务器版本保持一定的兼容性。如果两者版本差距过大, 可能会出现连接失败或功能异常的问题。这在企业统一部署时需要考虑。
- **复杂的网络环境:** 在一些有严格防火墙、特殊网络代理或复杂单点登录 (SSO) 配置的企业环境中, 配置桌面客户端使其成功连接到服务器, 有时会比直接在浏览器中访问更麻烦。

### 它的优势 (即, 它不被什么所限制)

为了更全面地看待问题, 了解它的优势也很重要, 这些优势恰恰解决了你之前遇到的问题:

- **剪贴板访问能力更强:** 这是它相对于浏览器版本的**巨大优势**。作为一个桌面应用, 它对系统剪贴板的访问权限比浏览器中的网页要高。因此, 从 **Word 或蓝凌KK**等桌面软件复制带有本地文件路径的图片时, **桌面版编辑器通常能够成功识别并直接读取图片数据进行上传**, 从而完美解决了你最初遇到的“占位符”问题。
- **本地字体支持:** 它可以直接使用你操作系统中安装的所有字体, 而浏览器版只能使用服务器提供和浏览器支持的 Web 字体。
- **无需担心浏览器兼容性:** 你不必担心用户的浏览器版本、插件冲突或缓存问题, 所有用户都使用统一的客户端环境。

总结来说, `ONLYOFFICE Desktop Editors` 的主要限制在于它无法像 Microsoft Office 那样与操作系统进行“像素级”的深度原生集成, 并且其插件生态系统尚在发展。然而, 在**核心的文档编辑和实时协同功能上**, 它与浏览器版保持了高度一致, 并额外提供了更强大的本地交互能力 (如剪贴板), 是 `DocumentServer` 一个非常出色的配套客户端。

Electron 的核心价值就是**打破浏览器的安全沙箱**, 让你能够用 Web 技术 (HTML, CSS, JavaScript) 构建的应用, 拥有几乎和原生应用 (用 C++, C#, Swift 编写的程序) 同等的能力。

当你的代码在 Electron 环境中运行时, 它就不再是“一个网页”, 而是“一个被信任的、安装在用户电脑上的应用程序”。这就解锁了海量的、浏览器端出于安全考虑而绝对禁止的能力。

以下是 Electron 能做到，而标准浏览器绝对不能或受限做到的核心能力：

## 1. 完全的本地文件系统访问

这是最根本的区别。

- **Electron:**

- 使用 Node.js 的 `fs` (File System) 模块，可以**自由地**、**静默地**读取、写入、修改、删除用户硬盘上任何有权限访问的文件和目录。
- 可以列出目录内容、创建文件夹、监控文件变化。
- **示例:** Visual Studio Code (一个典型的 Electron 应用) 可以读取你整个项目文件夹的内容，并在你保存文件时（如 `Ctrl+S`）立即将内容写入磁盘，这一切都无需弹窗询问。

```
const fs = require('fs');
// 直接读取 c 盘根目录下的一个配置文件
const config = fs.readFileSync('C:\\config.json', 'utf8');
// 直接向用户的“文档”文件夹写入日志
fs.writeFileSync('C:\\Users\\User\\Documents\\log.txt', 'Application started.');
```

- **浏览器:**

- **绝对无法主动访问本地文件。**
- 所有文件访问都必须由用户通过 `<input type="file">` 对话框或拖拽操作**明确发起**。
- 只能读取用户**主动选择**的单个或多个文件，且无法获取文件的真实路径。

## 2. 与操作系统的深度集成

- **Electron:**

- **自定义顶部菜单:** 可以创建和控制应用顶部的原生菜单栏（文件、编辑、视图等）。
- **系统托盘/菜单栏图标:** 可以在 Windows 的系统托盘或 macOS 的菜单栏中创建图标，并附加右键菜单。
- **原生对话框:** 可以调用操作系统的原生对话框，如“打开文件”、“保存文件”、“错误提示”等，而不是浏览器丑陋的 `alert()` / `confirm()`。
- **全局快捷键:** 可以注册一个在**整个操作系统层面**都生效的快捷键，即使你的应用不在前台或最小化了也能响应（例如，音乐播放器的全局播放/暂停键）。

```
const { app, Menu, Tray, dialog, globalShortcut } = require('electron');
// ... 创建托盘、菜单、注册全局快捷键 ...
```

- **浏览器:**

- 完全无法控制浏览器本身的菜单和界面。
- 无法创建系统托盘图标。
- 无法注册全局快捷键。

### 3. 无限制的网络请求

- **Electron:**
  - **没有 CORS (跨源资源共享) 限制:** 可以从你的应用直接向任何域名、任何 API 发送网络请求，不会被浏览器的同源策略阻止。
  - **可以作为服务器:** 可以使用 Node.js 的 `http` 或 `net` 模块，在本地启动一个 TCP 或 HTTP 服务器，监听特定端口。
  - **支持原始套接字 (Raw Sockets):** 可以进行底层的 TCP 和 UDP 通信。
  - **示例:** 一个本地的 API 调试工具（如 Postman 的早期版本）可以向任何服务发送请求。一个 P2P 应用可以直接与其他客户端建立连接。
- **浏览器:**
  - 严格受限于 CORS 策略。除非目标服务器明确允许，否则无法向不同源的地址发送 `fetch` 或 `XMLHttpRequest` 请求。
  - 无法创建服务器或使用原始套接字。

### 4. 任意的窗口管理

- **Electron:**
  - 可以创建无边框、透明、任意形状的窗口。
  - 可以精确控制窗口的大小、位置，并使其置顶。
  - 可以创建多个窗口并让它们相互通信。
  - **示例:** 很多游戏启动器 (Launcher) 或桌面小部件 (Widget) 都是用 Electron 实现的，它们拥有完全自定义的外观。
- **浏览器:**
  - 只能在浏览器提供的标准矩形标签页或弹窗中运行，外观无法自定义。

### 5. 执行原生代码和命令行工具

- **Electron:**
  - **Node.js Addons:** 可以加载和调用使用 C++ / Rust 等语言编写的二进制模块，用于执行性能敏感的计算或复用已有的原生库。
  - **子进程 (Child Process):** 可以使用 Node.js 的 `child_process` 模块来执行系统中的任何命令行工具（如 `git`, `ffmpeg`, `python` 脚本）并获取其输出。

```
const { exec } = require('child_process');
// 调用 git 命令获取版本号
exec('git --version', (error, stdout, stderr) => {
  console.log(stdout);
});
```

- **浏览器:**
  - 无法直接调用原生代码（WebAssembly 是一个受限的替代方案）。



- 绝对无法执行本地的命令行程序。

## 总结对比

| 能力    | Electron (基于 Node.js + Chromium) | 标准浏览器                     |
|-------|----------------------------------|---------------------------|
| 文件系统  | ✅ 完全读/写/执行权限                     | ❌ 只能访问用户手动选择的文件           |
| 原生 UI | ✅ 自定义菜单、托盘、对话框                   | ❌ 无法控制浏览器 UI              |
| 全局快捷键 | ✅ 支持                             | ❌ 不支持                     |
| 网络    | ✅ 无 CORS 限制，可作服务器                | ❌ 受 CORS 限制，只能作客户端        |
| 窗口    | ✅ 任意形状、大小、透明                     | ❌ 标准矩形窗口                  |
| 原生代码  | ✅ 可调用 C++ 模块和命令行                 | ❌ 不支持 (WebAssembly 是间接方式) |
| 硬件访问  | ✅ 可通过 Node 模块访问串口/USB            | ❌ 极度受限 (WebUSB/WebSerial) |

总之，Electron 就是把 **Web** 开发的便利性和 **桌面开发的强大能力** 结合了起来，让你用熟悉的工具链去构建功能不受限的跨平台桌面应用。

## 用新版本的Electron开发的应用，能否支持在window's7操作系统下使用

直接的答案是：不，使用新版本的 Electron 开发的应用，无法在 Windows 7 操作系统下使用。

这是一个硬性限制，背后有非常明确的原因和版本分界线。

### 为什么不支持？

这个限制的根源不在于 Electron 本身，而在于它的核心依赖——**Chromium**。

- Chromium 停止支持:** Google 在 2023 年初发布的 **Chromium 109** 是最后一个支持 Windows 7 和 Windows 8/8.1 的版本。从 **Chromium 110** 开始，其构建过程就依赖于 Windows 10 或更高版本才有的系统 API 和库。
- Electron 的跟随:** Electron 的核心就是 Chromium。当 Chromium 放弃对一个旧操作系统的支持后，Electron 别无选择，也必须放弃。继续支持将意味着巨大的、不可行的维护成本，并且无法获得来自上游 Chromium 的安全更新。
- 微软官方停止支持:** 微软早已在 **2020 年 1 月 14 日** 停止了对 Windows 7 的主流支持，并在 **2023 年 1 月 10 日** 停止了最后的扩展安全更新（ESU）。这意味着该操作系统已不再接收任何安全补丁，在上面运行任何联网应用都存在极大的安全风险。

### 关键的版本分界线

你需要非常清楚地记住这个版本号：

- **Electron 22.x:** 这是最后一个支持 Windows 7 / 8 / 8.1 的 Electron 版本系列。
- **Electron 23.x 及之后的所有版本:** 不再支持 Windows 7 / 8 / 8.1。

## 这对你作为开发者意味着什么？

如果你有必须支持 Windows 7 用户的需求，你将面临一个艰难的选择：

### 选项 1：坚持使用旧版本 (强烈不推荐)

你可以选择将你的项目“钉死”在 `Electron 22.x` 版本上进行开发和维护。

- **优点:**
  - 你的应用可以在 Windows 7 上运行。
- **缺点 (非常致命):**
  - **巨大的安全风险:** 你将永远无法获得 `Electron`, `Chromium`, `Node.js` 的后续安全更新。一旦这些组件的旧版本中发现任何严重漏洞，你的应用和你的用户将完全暴露在风险之下。
  - **功能停滞:** 你无法使用 Electron 23 之后所有的新功能、性能优化和 Bug 修复。
  - **无法使用新的 Web API:** 你将被困在 `Chromium 108` 的功能集里，无法使用此后浏览器支持的任何新 CSS 特性或 JavaScript API。
  - **生态系统兼容性问题:** 许多新的第三方 Node.js 模块或前端库可能会要求更高版本的 Node.js 或 Electron，你将无法使用它们。

### 选项 2：放弃支持 Windows 7 (官方推荐的最佳实践)

对于所有新项目和正在积极维护的项目，这是唯一合理的选择。

- **优点:**
  - **安全:** 你可以持续更新 Electron，及时获取所有安全补丁。
  - **现代:** 你可以使用最新的 Electron、Chromium 和 Node.js 功能，提升应用质量和开发效率。
  - **易于维护:** 你可以自由地使用最新的第三方库和工具。
- **缺点:**
  - 你将失去仍在使用 Windows 7 的那部分用户。

## 总结

| Electron 版本 | Windows 7 支持情况 | 建议                                      |
|-------------|----------------|-----------------------------------------|
| v22.x 及更早版本 | ✔ 支持           | 仅用于维护无法升级的老旧项目，存在严重安全隐患， <b>强烈不推荐</b> 。 |
| v23.x 及更新版本 | ✘ 不支持          | 所有新项目和积极开发项目的 <b>唯一正确选择</b> 。           |



总而言之，由于整个行业（从微软到谷歌）都已经放弃了 Windows 7，继续为其开发应用是一条充满安全风险且难以为继的道路。最佳实践是向用户说明情况，并要求他们升级到一个受支持的操作系统（Windows 10 或更高版本）。